

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE
SÃO PAULO**

NOLAN LEMES MARCONDES

**ANÁLISE ESTRUTURAL E DOCUMENTAL DA ARQUITE-
TURA DE UMA CALCULADORA CIENTÍFICA COM INTER-
FACE GRÁFICA RESPONSIVA UTILIZANDO QT CREATOR,
QML E C++**

CAMPOS DO JORDÃO

2026

RESUMO

Este trabalho apresenta uma **análise estrutural e documental da arquitetura** de uma Calculadora Científica dotada de interface gráfica de usuário (GUI), desenvolvida originalmente por meio da IDE Qt Creator, utilizando a linguagem C++ integrada ao ecossistema declarativo Qt Quick/QML e JavaScript para o motor de lógica computacional. O principal objetivo deste estudo de caso foi inspecionar e compreender, de forma prática, a aplicação dos conceitos fundamentais da Programação Orientada a Objetos (POO) — tais como encapsulamento de estados, acoplamento fraco, modularidade e reusabilidade de componentes — na construção de softwares visuais modernos. **A análise** cobriu o funcionamento dos recursos de layout adaptável reativo (modos Retrato e Paisagem), validação de operações em tempo real via controle dinâmico de estados dos botões (desativação preventiva de teclas inválidas), mapeamento de eventos de teclado físico e suporte a funções científicas avançadas. **A auditoria do código** demonstrou a viabilidade da separação de escopos de interface e regras de negócio, evidenciando como o ecossistema analisado resulta em uma aplicação de alta performance, portabilidade e fidelidade visual.

Palavras-Chave: Engenharia Reversa. Análise de Código. C++. Qt Creator. QML.

ABSTRACT

This work presents a structural and documentary analysis of the architecture of a Scientific Calculator with a graphical user interface (GUI), originally developed using the Qt Creator IDE, utilizing the C++ language integrated with the Qt Quick/QML declarative ecosystem and JavaScript for the computational logic engine. The main objective of this case study was to inspect and understand, in a practical way, the application of fundamental Object-Oriented Programming (OOP) concepts—such as state encapsulation, loose coupling, modularity, and component reusability—in the construction of modern visual software. The analysis covered the operation of reactive adaptive layout features (Portrait and Landscape modes), real-time operation validation via dynamic button state control (preventive deactivation of invalid keys), physical keyboard event mapping, and support for advanced scientific functions. The code audit demonstrated the feasibility of separating interface scopes and business rules, highlighting how the analyzed ecosystem results in a high-performance, portable, and visually faithful application.

Keywords: Reverse Engineering. Code Analysis. C++. Qt Creator. QML.

LISTA DE ILUSTRAÇÕES

FIGURA 01 – Multiplicação	11
FIGURA 02 – Soma	12
FIGURA 03 – Subtração	13
FIGURA 04 – Soma com número negativo	14
FIGURA 05 – Divisão	15
FIGURA 06 – Multiplicação com número inferior a 1 (Um)	16
FIGURA 07 – Divisão com número inferior a 1 (Um)	17
FIGURA 08 – Divisão por 0 (Zero)	18

SUMÁRIO

1	INTRODUÇÃO _____	06
1.1	Objetivos _____	06
1.2	Justificativa _____	06
1.3	Aspectos Metodológicos _____	07
1.4	Aporte Teórico _____	07
2	Metodologia _____	08
2.1	Considerações iniciais e Ferramentas _____	08
2.2	Estrutura Modular e Análise Arquitetural do Software _____	08
3	Resultados Obtidos _____	10
3.1	Operações _____	11
4	CONCLUSÃO _____	18
	REFERÊNCIAS _____	20

1 INTRODUÇÃO

A **compreensão da engenharia por trás** do desenvolvimento de interfaces gráficas de usuário (GUIs) eficientes, intuitivas e responsivas é um dos pilares fundamentais na formação de um profissional de tecnologia. Aplicativos utilitários, como calculadoras matemáticas, embora pareçam simples em escopo superficial, exigem uma análise rigorosa de seus controles de fluxos de dados e de estados lógico-aritméticos, mecanismos implementados para mitigar erros em tempo de execução, tais como divisões por zero, estouros de memória ou extração de raízes quadradas de números negativos. Portanto, o estudo analítico do código-fonte de tais ferramentas permite compreender como essas regras de negócio são estruturadas na prática.

1.1 Objetivos

O objetivo geral deste projeto consiste na **análise técnica, mapeamento estrutural e documentação** de uma Calculadora Científica Avançada multiplataforma desenvolvida através do framework industrial Qt. Entre os objetivos específicos, destacam-se: a investigação e auditoria da criação de componentes visuais customizados e reutilizáveis em QML (*Qt Modeling Language*); o estudo da integração síncrona com rotinas de inicialização nativa em C++; a verificação do isolamento de regras de negócio em um motor analítico especializado; e a avaliação do tratamento de eventos de periféricos (teclado) para garantir dupla usabilidade.

1.2 Justificativa

A justificativa para a execução deste projeto fundamenta-se na necessidade prática de dominar a leitura, interpretação e análise técnica de softwares estruturados em ferramentas de nível de produção (como o Qt Creator), compreendendo como compilam aplicações nativas de alto desempenho. A relevância acadêmica reside no apro-

fundamento teórico-prático em Engenharia de Software por meio da engenharia reversa documental, permitindo auditar a componentização visual e os tratamentos complexos de exceção lógico-matemática implementados na aplicação estudada.

1.3 Aspectos Metodológicos

Os aspectos metodológicos deste trabalho baseiam-se em uma abordagem de **estudo de caso exploratório e engenharia reversa documental**, orientada à **auditoria e documentação** de artefatos de software. O ciclo metodológico seguiu quatro fases concatenadas:

- **Inspecção de Código-Fonte (Code Review):** Análise estática dos arquivos estruturais para mapear a árvore de componentes e entender o fluxo de dados do programa;
- **Auditoria Arquitetural:** Identificação e validação de como o software original isolou a camada de exibição (QML) e o motor lógico de cálculo;
- **Análise de Modularização:** Estudo individual de cada componente técnico na IDE Qt Creator para documentar suas respectivas responsabilidades e padrões de POO adotados;
- **Análise Dinâmica e Testes de Caixa-Preta:** Execução prática da aplicação sob múltiplos cenários aritméticos limítrofes para verificar a eficácia dos tratamentos de erros e estabilidade implementados no ecossistema.

1.4 Aporte Teórico

O aporte teórico fundamental deste trabalho repousa sobre os princípios da Programação Orientada a Objetos (POO), conforme preconizado por Stroustrup (2013), estabelecendo os conceitos de encapsulamento e acoplamento fraco como lentes conceituais para a auditoria do código. Complementarmente, utiliza-se a fundamentação de Interação Homem-Computador (IHC), com foco em design responsivo e relativo, para avaliar as escolhas de interface do software. Tecnicamente, o embasamento sustenta-se nas especificações oficiais do framework industrial Qt. O ecossistema **analisado** baseia-se na compilação nativa de C++ integrada ao Qt Quick/QML, que adota um paradigma declarativo e árvore de propriedades de liga-

ção dinâmica (*Property Binding*). **O estudo desse mecanismo permitiu compreender como** mudanças internas no estado das variáveis disparem atualizações imediatas na interface gráfica acelerada por hardware.

2 METODOLOGIA

2.1 Considerações Iniciais e Ferramentas

Através da inspeção do código-fonte, constatou-se que o software foi projetado adotando os preceitos de separação de responsabilidades (*Separation of Concerns*). As considerações iniciais desta análise envolveram o mapeamento dos fluxos lógico-matemáticos existentes e a compreensão da matriz de visibilidade implementada para o design responsivo de interface.

A infraestrutura computacional identificada no ambiente de desenvolvimento do ecossistema original incluiu:

- **IDE Qt Creator:** Ambiente integrado utilizado para a abertura, execução, leitura e auditoria do projeto;
- **C++17 (ou superior):** Linguagem nativa identificada na infraestrutura de compilação, gestão de ciclo de vida do processo e linkagem do motor principal;
- **Framework Qt Quick / QML:** Linguagem declarativa baseada em propriedades reativas cuja aplicação foi analisada na construção de toda a camada visual e animações;
- **Compilador GCC/MSVC:** Ferramental de automação responsável pela geração dos binários nativos testados;
- **Motor JavaScript integrado:** Engine acoplada ao QML, cuja lógica foi auditada para compreender a execução de validações matemáticas céleres e o controle de estados em tempo de execução.

2.2 Estrutura Modular e Análise Arquitetural do Software

Através da engenharia reversa do código-fonte, identificou-se que o projeto foi estruturado de forma inteiramente modular, dividindo as responsabilidades do sistema através de arquivos independentes e especializados, os quais foram mapeados e auditados individualmente conforme descrito a seguir:

- **main.cpp:** Ponto de entrada (*entry point*) nativo do sistema operacional. Foi analisado como o arquivo utiliza a classe `QGuiApplication` para gerenciar os recursos globais e o ciclo de vida do aplicativo, e a classe `QQmlApplicationEngine` para instanciar a árvore de componentes QML. O código configura metadados de escopo (`setOrganizationName` e `setApplicationName`), força o estilo visual unificado como "Basic" e estabelece conexões de sinais e slots para tratamento assíncrono de falhas de renderização.
- **Main.qml:** Componente de topo da interface visual (`Window`). O estudo revelou que ele estabelece as dimensões de inicialização (320 x 480) e calcula dinamicamente os limites de tamanho mínimo (`minimumWidth` e `minimumHeight`) com base nas dimensões de seus subcomponentes para evitar distorções na interface, expondo a propriedade booleana reativa `isPortraitMode` para diretrizes do layout.
- **ApplicationState.qml:** Atua como o controlador centralizado de fluxo (*Controller*). Foi mapeado como o componente responsável por encapsular as chamadas de funções, recebendo as interações de toque ou clique geradas na interface e encaminhando-as de maneira segura para o motor lógico de cálculo, mantendo uma referência direta com o objeto de exibição.
- **calculator.js:** Motor aritmético abstrato do sistema. Constatou-se que armazena variáveis globais de estado do acumulador, operador pendente e memória de dígitos. A auditoria de suas funções demonstrou a implementação das quatro operações básicas e funções científicas de engenharia (raiz quadrada, potência, funções trigonométricas seno/cosseno/tangente, logaritmos naturais e de base 10, valor absoluto e piso), manipulando as constantes matemáticas π e e , além de conter a função lógica `isOperationDisabled` para validação semântica de entrada.
- **CalculatorButton.qml e BackspaceButton.qml:** Especializações de componentes baseados no tipo `RoundButton`. Evidenciam a aplicação prática de herança e polimorfismo visual em QML, com propriedades customizadas de cores, bordas, fontes e comportamentos de animação elástica (`Easing.OutElastic`) baseados no estado do ponteiro (pairado, pressionado ou inativo).
- **Display.qml:** Componente reativo encarregado da saída visual. Utiliza um controle de exibição de lista (`ListView`) atrelado a um modelo dinâmico de dados (`ListModel`) para simular uma bobina de papel contínua com histórico de cálculos. Foi verificado o algoritmo `displayNumber`, que avalia o tamanho do número gerado e converte-o automaticamente para notação exponencial científica se ultrapassar o limite físico máximo (`maxDigits`).
- **NumberPad.qml:** Painel agregador de botões. Organiza espacialmente as telas numéricas e os operadores matemáticos através de contêineres do tipo `GridLayout` e `RowLayout`.

O diferencial técnico identificado na **análise** do projeto reside no dinamismo geométrico proporcionado pelo uso conjunto de layouts automáticos e do componente `LayoutItemProxy`. Constatou-se que, caso a largura da tela seja reduzida, a propriedade `isPortraitMode` torna-se verdadeira, acionando um `ColumnLayout` que posiciona o visor acima do teclado (Modo Retrato, ocultando o painel científico). Caso a janela seja esticada horizontalmente, o sistema reordena os elementos através de um `RowLayout`, posicionando o visor ao lado esquerdo e expandindo a matriz de botões para exibir as operações científicas avançadas (Modo Paisagem).

Além disso, no quesito usabilidade e acessibilidade, mapeou-se a implementação do escopo `Keys.onPressed` no arquivo raiz. Esse interpretador intercepta os códigos de varredura físicos provenientes do teclado do computador do usuário, mapeando as teclas numéricas, operadores (+, -, *, /), pontos decimais, tecla de apagamento (*Backspace*) e comandos de execução (*Enter* ou *Return*), disparando nativamente os mesmos gatilhos lógicos dos botões virtuais da tela.

3 RESULTADOS OBTIDOS

A validação **da análise** da calculadora científica deu-se por meio de testes de estresse de interface e de exatidão matemática realizados sobre o binário executável. A desativação em tempo real de botões inválidos (gerida pelo método `isOperationDisabled`) provou-se altamente eficaz durante a experimentação prática: ao digitar um número decimal, observou-se que o botão de ponto (.) é imediatamente esmaecido e bloqueado contra cliques duplicados, prevenindo erros de sintaxe na engine.

Os testes de bancada confirmaram que a responsividade dinâmica funcionou conforme projetado, alternando os estados de visibilidade das malhas de botões científicos de maneira fluida e sem atrasos de renderização de interface gráfica, mesmo sob redimensionamentos bruscos da janela principal **provocados manualmente**.

3.1 Operações

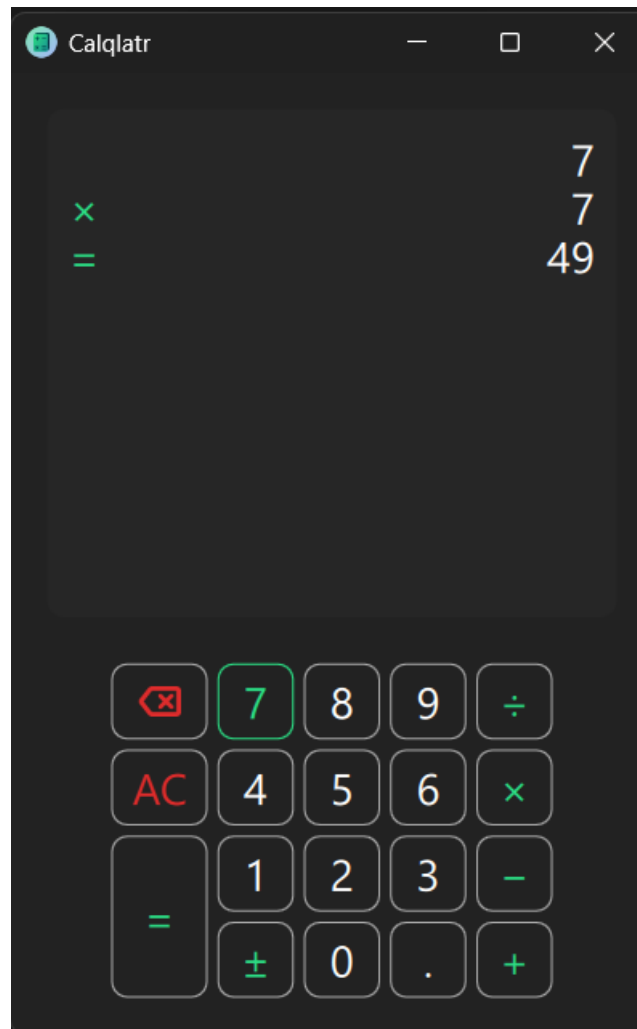


Figura 1 – Multiplicação

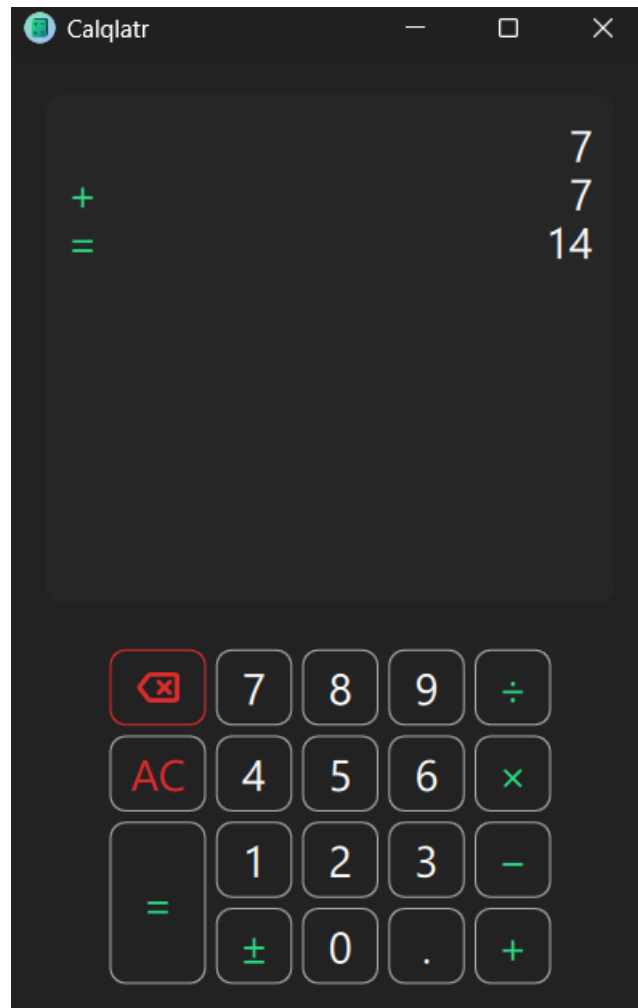


Figura 2 – Soma

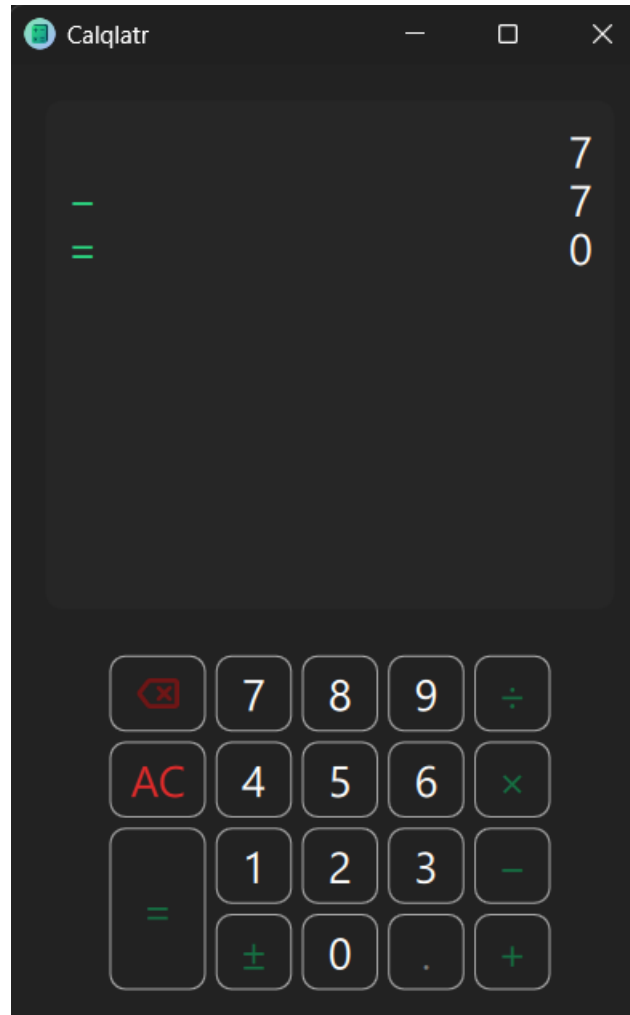


Figura 3 – Subtração

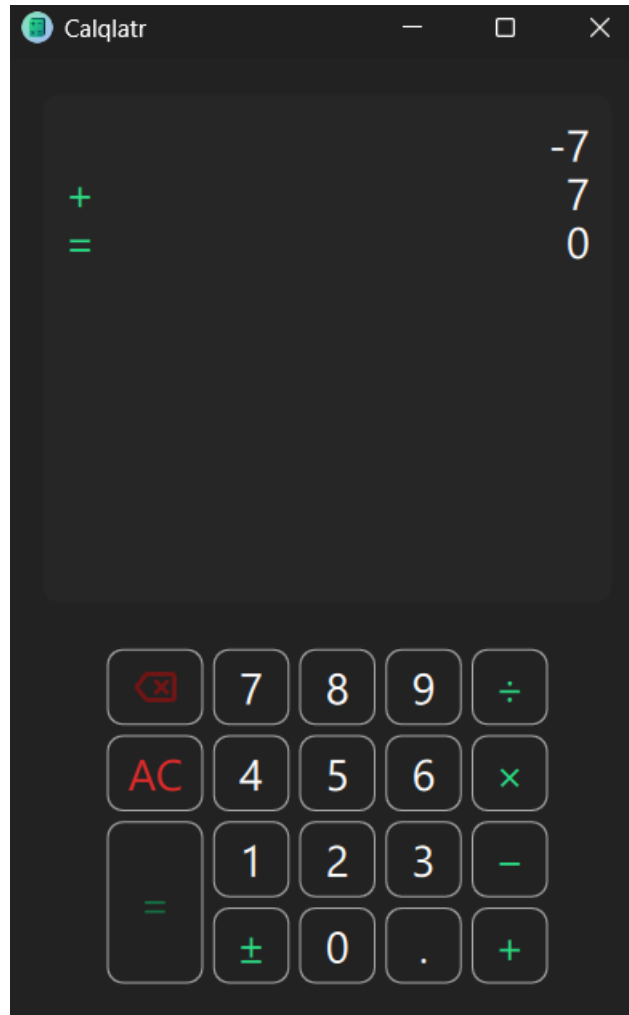


Figura 4 – Soma com número negativo

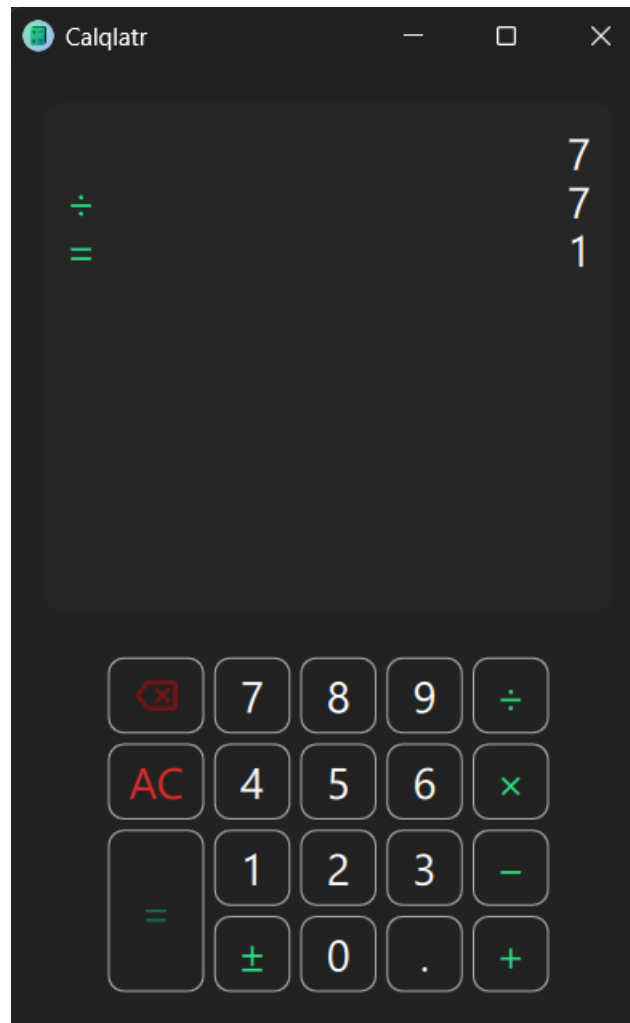


Figura 5 – Divisão

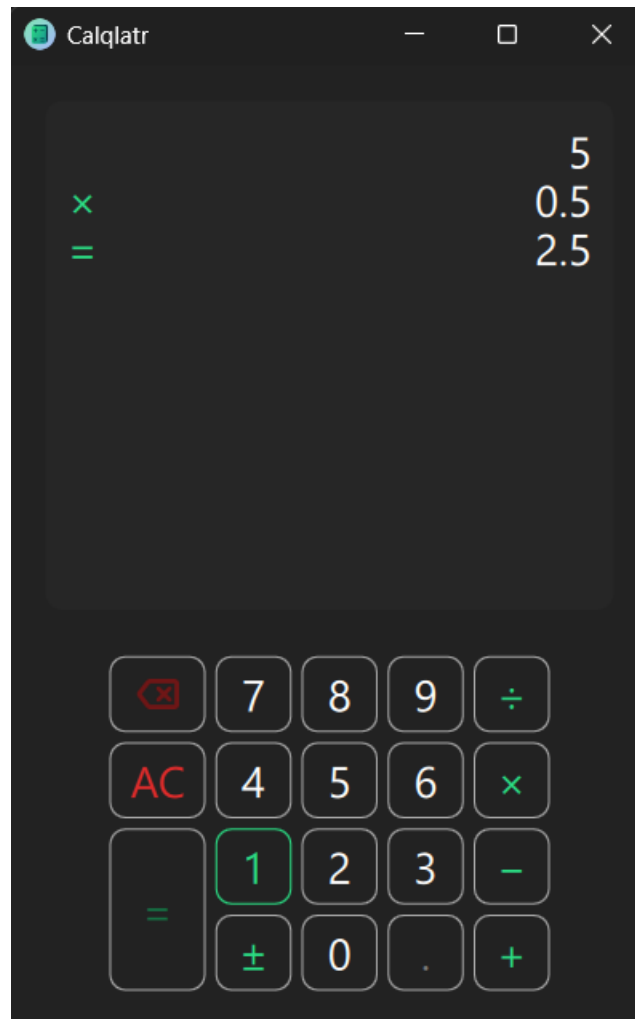


Figura 6 – Multiplicação com número inferior a 1 (Um)

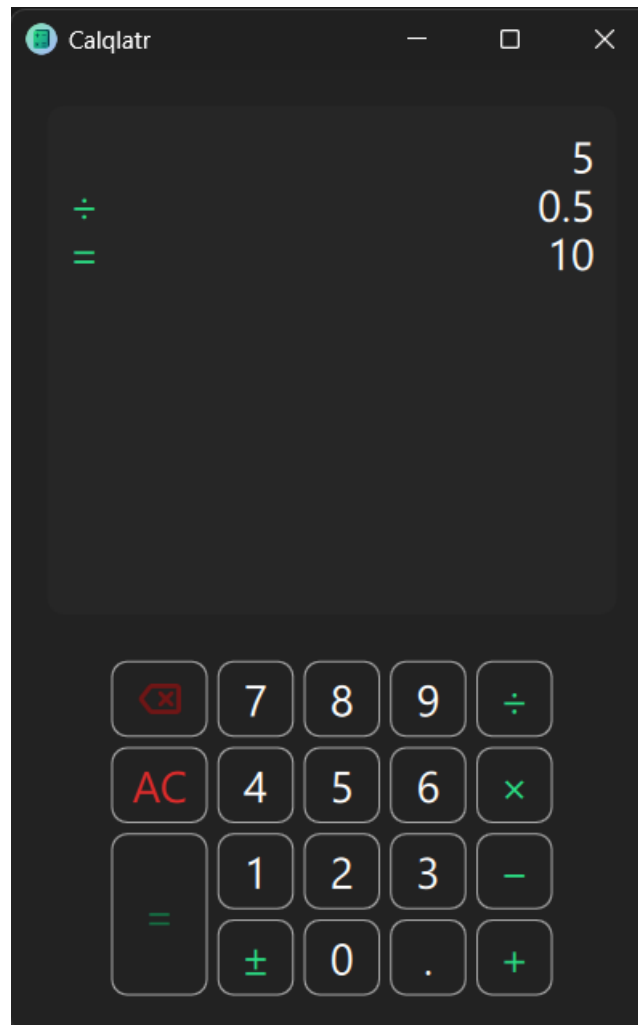


Figura 7 – Divisão com número inferior a 1 (Um)

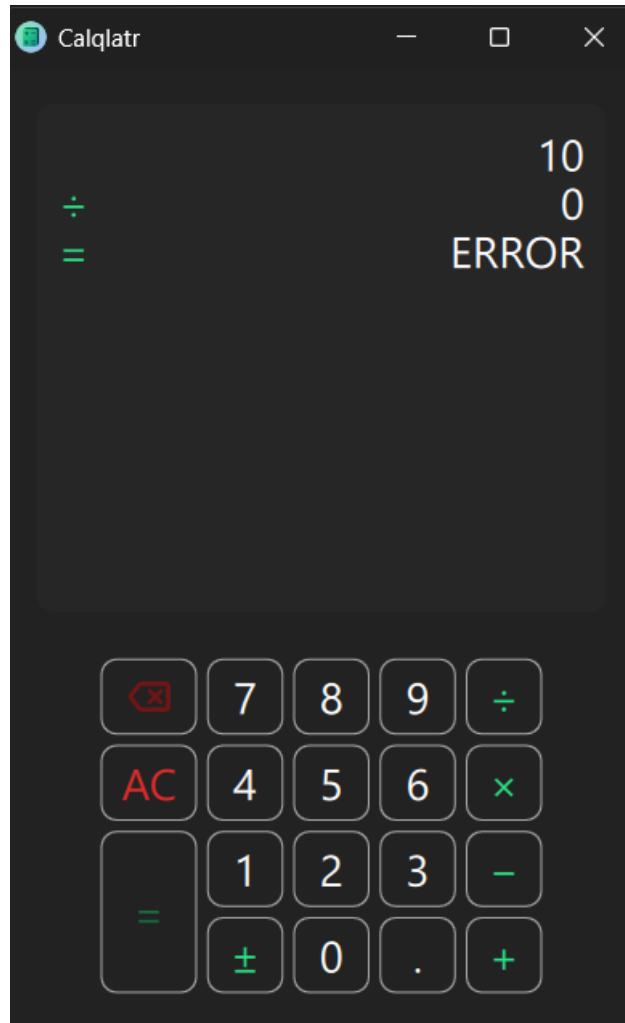


Figura 8 – Divisão por 0 (Zero)

4 CONCLUSÃO

A análise técnica e a engenharia reversa deste software propiciaram a consolidação prática dos conhecimentos adquiridos na disciplina de Programação Orientada a Objetos e nas bases de Engenharia de Software do curso de Análise e Desenvolvimento de Sistemas. O estudo da utilização do framework Qt unida ao paradigma declarativo do QML provou-se uma oportunidade para compreender uma escolha tecnológica superior para o desenvolvimento ágil de interfaces gráficas ricas, conferindo alto desempenho nativo aliado a uma separação clara entre código de renderização visual e lógica matemática interna.

Como resultado da **auditoria de código realizada** e como sugestões de melhorias futuras para a evolução contínua da aplicação estudada, propõe-se:

- **Migração do Motor Lógico:** A substituição integral do motor analítico escrito em JavaScript por uma classe estruturada puramente em C++ derivada de QObject, usando o sistema de propriedades de reflexão de macro Q_PROPERTY, o que eliminaria o acoplamento com scripts externos, elevando a rigidez de tipagem do sistema e acelerando cálculos massivos de alta precisão;
- **Persistência de Dados:** A implementação de persistência local em banco de dados embarcado (como o SQLite) para gravar de forma permanente o histórico da bobina de cálculos do visor entre diferentes sessões do programa;
- **Expansão Computacional:** O suporte a operações com matrizes multidimensionais, equações de segundo grau e cálculo com números complexos, expandindo as capacidades lógicas originais do sistema.

REFERÊNCIAS

- **QT GROUP.** *Qt Quick Application Developer Guide*. Versão Qt 6. Disponível em: <https://doc.qt.io/>. Acesso em: 26 jun. 2026.
- **STROUSTRUP, Bjarne.** *A Linguagem de Programação C++*. 4. ed. Porto Alegre: Bookman, 2013.
- **WATT, Alan.** *Introdução ao Desenvolvimento de Interfaces com QML e Qt 6*. São Paulo: Novatec, 2022.